UNIT II – FUNCTIONS AND CLASSES & OBJECTS IN C++

Complete Answers: Debugging and Pseudo-Code

Section A: Debugging Questions

Question 1: Dangling Reference in Return-by-Reference

Topic: Return by reference, scope and object lifetime

1. Lifetime of result and undefined behaviour

The local variable result is an automatic (stack) object created when getMaximumScaled is called. Its lifetime ends when the function returns. Returning int& does not extend that lifetime. The returned reference therefore refers to an object that no longer exists. Reading danglingRef later (including on the first output statement) has undefined behaviour: it may print a garbage value, appear to work, or cause a crash.

2. Why getArrayElement is safe

arr refers to an array supplied by the caller. The array is not a local array created inside getArrayElement, so its elements remain alive after the function returns (provided the caller's array remains in scope). Returning arr[index] by reference therefore refers to a live element. The caller must still pass a valid index.

3. Corrected code

```
#include <iostream>

int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    return maxVal * scaleFactor;       // Return by value
}

int& getArrayElement(int arr[], int index) {
    return arr[index];                 // Refers to caller-owned array
}

int main() {
    int x = 10, y = 20;
    int scaled = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << scaled << '\n';

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;
    std::cout << "Updated Array[1]: " << data[1] << '\n';
    std::cout << "Scaled Value: " << scaled << '\n';
    return 0;
}
```

4. Dry run and expected output

Step	Operation	Result / state

1	x = 10, y = 20	Inputs are 10 and 20
2	getMaximumScaled(10, 20, 2)	maxVal = 20; result = 40; returns a copy
3	scaled receives return value	scaled = 40; no dangling reference
4	getArrayElement(data, 1) = 999	data becomes {100, 999, 300}
5	Print values	40, 999, 40

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Value: 40
```

Question 2: Static Data Member Linkage and Member Function Scoping

Topic: Static data members and static member functions

1. Linker error

The in-class declaration static double totalBankReserves; declares a class-level member but, in the traditional out-of-class definition model used here, does not provide its storage definition. Add exactly one definition at namespace/file scope: double BankAccount::totalBankReserves = 0.0;. Without it, uses of the member can produce an undefined-reference linker error.

2. Compilation error

displayAudit is static and is not called for a particular BankAccount object. It has no implicit this pointer, so it cannot directly access accHolder or balance, which belong to individual instances. The static method may access the static totalBankReserves. To display a holder, use a non-static member method called through an object.

3. Corrected code

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;

public:
    BankAccount(const std::string& name, double initialDeposit)
        : accHolder(name), balance(initialDeposit) {
        totalBankReserves += initialDeposit;
    }

    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder
                  << ", Balance: $" << balance << '\n';
    }

    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $"
                  << totalBankReserves << '\n';
    }
```

```
};

double BankAccount::totalBankReserves = 0.0;

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();
    BankAccount::displayAudit();
    return 0;
}
```

4. Expected output

```
Account Holder: Alice, Balance: $5000
Account Holder: Bob, Balance: $3000
Total Reserves in Vault: $8000
```

Question 3: Function Overload Ambiguity via Default Arguments

Topic: Function overloading and default arguments

1. Why the one-argument call is ambiguous

computeVolume(5.0) can call the cuboid overload because width and height have defaults, and it can also call the one-parameter cube overload. Both are viable with one double argument, so neither is a better match; overload resolution is ambiguous.

2. Two-argument call

computeVolume(3.0, 7.0) is also ambiguous in the supplied code: it can match the cuboid overload (length and width, with default height) and the cylinder overload (radius and height). Both take two double arguments with exact matches. The comments in the question should therefore not be read as making this call unambiguous.

3. Refactoring

Give each geometric operation a distinct function name. This removes overload ambiguity while preserving the cuboid's optional dimensions.

4. Corrected code and output

```
#include <iostream>

double cuboidVolume(double length, double width = 1.0,
                    double height = 1.0) {
    return length * width * height;
}

double cubeVolume(double side) {
    return side * side * side;
}

double cylinderVolume(double radius, double height) {
    return 3.14159 * radius * radius * height;
```

```
}

int main() {
    std::cout << "Cube: " << cubeVolume(5.0) << '\n';
    std::cout << "Cuboid: " << cuboidVolume(5.0) << '\n';
    std::cout << "Cylinder: " << cylinderVolume(3.0, 7.0) << '\n';
    return 0;
}
```

```
Cube: 125
Cuboid: 5
Cylinder: 197.92
```

Cylinder calculation: 3.14159 × 3 × 3 × 7 = 197.92017, displayed as 197.92 under the default stream formatting.

Question 4: Array of Objects Missing Default Constructor

Topic: Arrays of objects and constructors

1. Constructor rule

When a class declares a constructor with parameters, the compiler does not implicitly generate a no-argument default constructor. Student batch[3] requires the default constructor to create each of the three elements, so the declaration fails because no accessible zero-argument constructor exists.

2. Two solutions

Technique A: Declare a default constructor, for example Student() : rollNumber(0), name("") {}. The array can then be created first and its elements assigned later.

Technique B: Initialize each array element directly with the parameterized constructor: Student batch[3] = { Student(101, "Aarav"), Student(102, "Bhavna"), Student(103, "Chirag") };. This does not require a default constructor.

3. Corrected program using Technique A

```
#include <iostream>
#include <string>

class Student {
private:
    int rollNumber;
    std::string name;

public:
    Student() : rollNumber(0), name("") {}

    Student(int r, const std::string& n)
        : rollNumber(r), name(n) {}

    void display() const {
        std::cout << "Roll: " << rollNumber
                  << ", Name: " << name << '\n';
    }
};

int main() {
```

```
    Student batch[3];

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i)
        batch[i].display();

    return 0;
}
```

4. Expected output

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration and Scope Resolution Flaws

Topic: Friend functions, forward declarations and non-member scope

1. Access to alertThreshold

A friend function is a non-member function. Friendship grants permission to access private members, but it does not provide an implicit object or this pointer. The function must use a specific TelemetryAuditor object, such as a.alertThreshold. Similarly, engine members must be qualified as e.temperature and e.rpm.

2. Correct invocation

A friend function is not a member of Engine, so v8.inspectDiagnostics(v8, audit) is invalid. Call it as an ordinary function: inspectDiagnostics(v8, audit).

3. Forward declaration

Engine's friend declaration names TelemetryAuditor before that class is defined. A forward declaration, class TelemetryAuditor;, tells the compiler that this is a class type and allows it to parse the function parameter declaration. The function body is defined after both classes are complete.

4. Corrected code

```
#include <iostream>

class TelemetryAuditor;  // Forward declaration

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    friend void inspectDiagnostics(const Engine& e,
                                   const TelemetryAuditor& a);
};
```

```
class TelemetryAuditor {
private:
   int alertThreshold;

public:
   explicit TelemetryAuditor(int threshold)
      : alertThreshold(threshold) {}

   friend void inspectDiagnostics(const Engine& e,
                                  const TelemetryAuditor& a);
};

void inspectDiagnostics(const Engine& e, const TelemetryAuditor& a) {
   if (e.temperature > a.alertThreshold) {
      std::cout << "Warning: Engine overheat!\n";
   } else {
      std::cout << "Engine operating nominally at "
                << e.rpm << " RPM.\n";
   }
}

int main() {
   Engine v8(105, 3200);
   TelemetryAuditor audit(100);
   inspectDiagnostics(v8, audit);
   return 0;
}
```

Expected output

```
Warning: Engine overheat!
```

Section B: Pseudo-Code Writing Questions

The following answers use the class, method, REF and scope-notation conventions requested in the question bank. Each trace uses the supplied sample values or states any additional sample values needed.

Question 1: Corporate Vault Management with Static Data and Functions

Topic: Static attributes and class methods

1. Structured pseudo-code

```
CLASS BranchOffice
   PRIVATE:
      branchID AS STRING
      branchName AS STRING
      currentBalance AS REAL
      STATIC totalVaultReserves AS REAL <- 0.0
      STATIC totalBranchesActive AS INTEGER <- 0

   PUBLIC:
      CONSTRUCTOR(id, name, openingDeposit)
         branchID <- id
         branchName <- name
         currentBalance <- openingDeposit
         totalVaultReserves <- totalVaultReserves + openingDeposit
```

```
        totalBranchesActive <- totalBranchesActive + 1
    END CONSTRUCTOR

    METHOD Deposit(amount AS REAL)
        IF amount > 0 THEN
            currentBalance <- currentBalance + amount
            totalVaultReserves <- totalVaultReserves + amount
        END IF
    END METHOD

    METHOD Withdraw(amount AS REAL) RETURNS BOOLEAN
        IF amount > 0 AND currentBalance >= amount THEN
            currentBalance <- currentBalance - amount
            totalVaultReserves <- totalVaultReserves - amount
            RETURN TRUE
        END IF
        RETURN FALSE
    END METHOD

    STATIC METHOD GetVaultSummary()
        OUTPUT "Active branches: ", totalBranchesActive
        OUTPUT "Total reserves: ", totalVaultReserves
    END METHOD

    STATIC METHOD GetGlobalLiquidityRatio(targetReserve AS REAL)
        IF targetReserve <= 0 THEN
            RETURN 0.0
        END IF
        RETURN totalVaultReserves / targetReserve
    END METHOD
END CLASS

MAIN
    CREATE Branch1 AS BranchOffice("B01", "North", 200000)
    CREATE Branch2 AS BranchOffice("B02", "South", 120000)

    Branch1.Deposit(150000)
    success <- Branch2.Withdraw(50000)

    BranchOffice.GetVaultSummary()
    ratio <- BranchOffice.GetGlobalLiquidityRatio(500000)
    OUTPUT "Withdrawal successful: ", success
    OUTPUT "Liquidity ratio: ", ratio
END MAIN
```

2. Trace table

Stage	Branch 1 balance	Branch 2 balance	Static reserves	Active branches
Initial after both constructors	200,000	120,000	320,000	2
Branch 1 deposits 150,000	350,000	120,000	470,000	2
Branch 2 withdraws 50,000	350,000	70,000	420,000	2

Final summary: 2 active branches and total reserves of 420,000. With a target reserve of 500,000, the liquidity ratio is 420,000 / 500,000 = 0.84 (84%). The withdrawal succeeds because Branch 2 has 120,000 before withdrawal.

Question 2: Matrix Mathematical Engine with Arrays in a Class and Friend Dot-Product

Topic: Two-dimensional arrays, scope resolution and friend functions

1. Structured pseudo-code

```
CLASS Matrix2D
   PRIVATE:
      grid AS ARRAY[3][3] OF REAL

   PUBLIC:
      METHOD Initialize()
      METHOD SetElement(row AS INTEGER, col AS INTEGER, value AS REAL)
      METHOD GetElement(row AS INTEGER, col AS INTEGER) RETURNS REAL
      METHOD Display()
      FRIEND FUNCTION ComputeCrossTrace(A AS Matrix2D, B AS Matrix2D)
         RETURNS REAL
END CLASS

METHOD Matrix2D::Initialize()
   FOR row <- 0 TO 2
      FOR col <- 0 TO 2
         grid[row][col] <- 0
      END FOR
   END FOR
END METHOD

METHOD Matrix2D::SetElement(row, col, value)
   IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
      grid[row][col] <- value
   ELSE
      OUTPUT "Error: matrix index out of bounds"
   END IF
END METHOD

METHOD Matrix2D::GetElement(row, col) RETURNS REAL
   IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
      RETURN grid[row][col]
   ELSE
      OUTPUT "Error: matrix index out of bounds"
      RETURN 0
   END IF
END METHOD

METHOD Matrix2D::Display()
   FOR row <- 0 TO 2
      FOR col <- 0 TO 2
         OUTPUT grid[row][col], " "
      END FOR
      OUTPUT NEWLINE
   END FOR
END METHOD

FUNCTION ComputeCrossTrace(A, B) RETURNS REAL
   sum <- 0
   FOR i <- 0 TO 2
```

```
        sum <- sum + A.grid[i][i] * B.grid[i][i]
    END FOR
    RETURN sum
END FUNCTION
```

2. Friend function trace

i	A[i][i]	B[i][i]	Product	Running sum
0	2	1	2 × 1 = 2	2
1	4	3	4 × 3 = 12	14
2	6	5	6 × 5 = 30	44

ComputeCrossTrace returns 44. It can access grid directly because it is declared as a friend in Matrix2D; it remains a non-member function.

Question 3: Student Academic Roster via Array of Objects

Topic: Arrays of objects and bubble sort

1. Structured pseudo-code

```
CLASS StudentRecord
    PRIVATE:
        rollNo AS INTEGER
        studentName AS STRING
        marks AS ARRAY[3] OF REAL
        percentage AS REAL
        grade AS CHARACTER

        METHOD CalculateResult()
            percentage <- (marks[0] + marks[1] + marks[2]) / 3
            IF percentage >= 85 THEN
                grade <- 'A'
            ELSE IF percentage >= 70 THEN
                grade <- 'B'
            ELSE IF percentage >= 50 THEN
                grade <- 'C'
            ELSE
                grade <- 'F'
            END IF
        END METHOD

    PUBLIC:
        CONSTRUCTOR()
            rollNo <- 0
            studentName <- ""
            marks[0] <- 0
            marks[1] <- 0
            marks[2] <- 0
            percentage <- 0
            grade <- 'F'
        END CONSTRUCTOR

        METHOD ReadData(r, name, m1, m2, m3)
            rollNo <- r
            studentName <- name
            marks[0] <- m1
            marks[1] <- m2
```

```
        marks[2] <- m3
        CALL CalculateResult()
    END METHOD

    METHOD GetPercentage() RETURNS REAL
        RETURN percentage
    END METHOD

    METHOD Display()
        OUTPUT rollNo, studentName, percentage, grade
    END METHOD
END CLASS

MAIN
    DECLARE students AS ARRAY[3] OF StudentRecord
    students[0].ReadData(1, "Anu", 90, 88, 92)   // 90
    students[1].ReadData(2, "Bala", 75, 70, 80)  // 75
    students[2].ReadData(3, "Charu", 60, 55, 65) // 60

    FOR pass <- 0 TO 1
        FOR j <- 0 TO 1 - pass
            IF students[j].GetPercentage() <
               students[j+1].GetPercentage() THEN
                TEMP <- students[j]
                students[j] <- students[j+1]
                students[j+1] <- TEMP
            END IF
        END FOR
    END FOR

    OUTPUT "Rank list:"
    FOR i <- 0 TO 2
        students[i].Display()
    END FOR
END MAIN
```

2. Bubble-sort trace

Sample percentages are calculated from the three marks supplied above: Anu = 90, Bala = 75, Charu = 60.

Pass / comparison	Compared percentages	Action	Order after action
Initial	—	—	Anu 90, Bala 75, Charu 60
Pass 1, j=0	90 and 75	No swap	Anu 90, Bala 75, Charu 60
Pass 1, j=1	75 and 60	No swap	Anu 90, Bala 75, Charu 60
Pass 2, j=0	90 and 75	No swap	Anu 90, Bala 75, Charu 60

The list is already in descending order, so no swaps occur. Final grades are Anu: A, Bala: B, and Charu: C.

Question 4: Flexible Logger with Overloaded Functions and Inline Helpers

Topic: Overloading, defaults and inline functions

1. Structured pseudo-code

```
INLINE FUNCTION FormatPrefix(prefixTag AS STRING) RETURNS STRING
   RETURN "[" + prefixTag + "] "
END FUNCTION

FUNCTION LogEvent(message AS STRING)
   OUTPUT FormatPrefix("INFO") + message
END FUNCTION

FUNCTION LogEvent(message AS STRING,
                  severityLevel AS INTEGER = 1,
                  timestamp AS STRING = "00:00:00")
   IF severityLevel = 1 THEN
      severity <- "LOW"
   ELSE IF severityLevel = 2 THEN
      severity <- "MEDIUM"
   ELSE IF severityLevel = 3 THEN
      severity <- "CRITICAL"
   ELSE
      severity <- "UNKNOWN"
   END IF
   OUTPUT "[" + severity + "] @ " + timestamp + " : " + message
END FUNCTION

FUNCTION LogEvent(errorCode AS INTEGER, systemModule AS STRING)
   hexCode <- CONVERT_TO_HEX(errorCode)
   OUTPUT "[HARDWARE_FAULT] Module: " + systemModule +
          " -> Error Code: 0x" + hexCode
END FUNCTION

MAIN
   LogEvent("System started")
   LogEvent("Temperature rising", 2)
   LogEvent("Sensor failure", 3, "14:35:20")
   LogEvent(255, "ADC")
END MAIN
```

2. Calls and default argument resolution

Call	**Selected overload**	**Resolved values / output**
LogEvent("System started")	One-argument message overload	[INFO] System started
LogEvent("Temperature rising", 2)	Message + severity + timestamp overload	severity=2 → MEDIUM; timestamp defaults to 00:00:00
LogEvent("Sensor failure", 3, "14:35:20")	Message + severity + timestamp overload	severity=3 → CRITICAL; explicit timestamp
LogEvent(255, "ADC")	Integer + module overload	0xFF in hexadecimal

The separate one-argument overload and the overload with defaulted parameters can create ambiguity in C++ if both are declared with these exact signatures. A compilable C++ design should remove that overlap (for example, make the second overload require at least two arguments, or use distinct names). The pseudo-code above follows the requested three conceptual overloads; for unambiguous implementation, revise the second signature to require severityLevel and keep timestamp defaulted.

Question 5: Hardware Register Bank with Return-by-Reference

Topic: Return-by-reference, array members and lvalue assignment

1. Structured pseudo-code

```
CLASS RegisterBank
   PRIVATE:
      registers AS ARRAY[8] OF INTEGER
      errorSink AS INTEGER

   PUBLIC:
      CONSTRUCTOR()
         FOR i <- 0 TO 7
            registers[i] <- 0
         END FOR
         errorSink <- -1
      END CONSTRUCTOR

      METHOD GetRegister(index AS INTEGER) RETURNS REF INTEGER
         IF index >= 0 AND index < 8 THEN
            RETURN REF registers[index]
         ELSE
            OUTPUT "Alert: register index out of bounds"
            RETURN REF errorSink
         END IF
      END METHOD

      METHOD DisplayState()
         FOR i <- 0 TO 7
            OUTPUT "Register[", i, "] = ",
                 TO_HEX(registers[i])
         END FOR
      END METHOD
END CLASS

MAIN
   CREATE bank AS RegisterBank

   bank.GetRegister(0) <- 170
   bank.GetRegister(1) <- 255

   readVal <- bank.GetRegister(0)
   OUTPUT "Read register 0: ", readVal

   bank.GetRegister(10) <- 999
   OUTPUT "Error sink: ", bank.GetRegister(-1)
   bank.DisplayState()
END MAIN
```

2. Why a function call can be on the left side of assignment

GetRegister returns a reference to an existing integer object, not a temporary copy. For a valid index, that reference aliases registers[index]. Thus bank.GetRegister(0) <- 170 means assign 170 to the actual first element of the internal array. For an invalid index, the function returns a reference to errorSink, so the attempted write changes only that fallback variable and does not access an out-of-range array element. The returned reference must always refer to an object whose lifetime continues after the method returns.